Dual-core machine learning accelerator for attention mechanism

Shreeya Ajay Bhonsle (A69040642) , Chi-Han Chiu (A69036037) , Nikhita Neelakanta (A69041742) , Ming-Yang Wu (A16504833) , Bing-Cheng Chiang (A69040193) , Wen-Chieh Lo (A69042225)

***Abstract*—This report presents the design and implementation of a dual-core machine learning accelerator tailored for attention mechanisms, a core component of modern transformer models. The architecture features an asynchronous clocking scheme with a dual-core setup to parallelize the matrix multiplication and normalization phases. Furthermore, a hardware-optimized sparsity-aware Softmax (SFP) engine is introduced, utilizing row-energy accumulation and cross-core sum exchange to execute accurate fixed-point normalization. We also outline a comprehensive step-by-step verification methodology that spans from single-core RTL design to full-chip hierarchical physical design (PnR) using a 65nm process. Architectural optimizations, such as repipelining the divider and adder tree, dramatically reduced the critical path delay from over 24 ns to under 1 ns, achieving zero timing violations. Finally, our design achieves fully functional gate-level simulations and successful tape-out-ready physical layouts.**

I. Introduction

Attention mechanisms have revolutionized natural language processing and computer vision by enabling models to dynamically weigh the importance of different input elements. At the core of the attention mechanism is the computation of attention scores, which determines how much focus a query should place on various keys. This is mathematically expressed as multiplying a Query matrix (Q) with a transposed Key matrix (K) to obtain raw attention scores. These raw scores are then normalized using a Softmax function, converting them into a valid probability distribution where all weights sum to one. Finally, the normalized attention weights are multiplied by a Value matrix (V) to produce the final output context vectors.

However, the associated matrix multiplication (MatMul) and Softmax normalization operations pose significant computational and memory bandwidth challenges. The Softmax function, in particular, requires global awareness of the entire row (or sequence) to compute the normalizing sum, creating a bottleneck in deeply pipelined hardware. To address these bottlenecks, this project proposes a dual-core hardware accelerator designed specifically for the attention mechanism.

The proposed design leverages two independent cores communicating via asynchronous FIFOs, allowing flexible scheduling and increased throughput. To efficiently handle the Softmax operation, we developed a Sparsity-Aware Softmax (SFP) engine that skips redundant computations for zero or near-zero data, thereby saving power and cycles.

Due to the immense complexity of the dual-core asynchronous design, a rigorous, step-by-step implementation and verification strategy was adopted. This incremental approach—ranging from basic single-core synthesis to full-chip hierarchical place-and-route (PnR)—ensured functional correctness at every stage. This report details both the theoretical architecture of the accelerator and the practical milestones achieved during its physical implementation.

II. Proposed Architecture

A. Dual-Core System & Asynchronous Clocking

To maximize throughput, the accelerator is partitioned into two distinct processing cores (Core 0 and Core 1). Each core operates in its own clock domain (`clk0` and `clk1`) to simulate realistic system-on-chip (SoC) environments where different functional blocks might run at different frequencies. Data and control signals are securely transferred between the two clock domains using asynchronous FIFOs (Async FIFO). This cross-domain communication (CDC) ensures that matrix chunks processed by one core can be synchronized with the other core without meta-stability issues, particularly during the global sum accumulation required for Softmax normalization.

B. Core Controller & Data Flow

Each core features an independent controller and a memory map (`reg_map`) to handle top-level configurations. The controller manages the data flow between the Query Memory (QMEM), Key Memory (KMEM), the MAC array, and the SFP engine based on the defined operation mode (`op_mode`):

- **Mode 1 (MatMul Only):** The core multiplies data from QMEM and KMEM using the MAC array, and directly stores the result into the Partial Sum Memory (PMEM).
- **Mode 0 (MatMul + Normalization):** Data from QMEM and KMEM are multiplied, and the resulting partial sums are fed into the SFP engine. The normalized outputs are then written back to either PMEM or KMEM, dictated by `sfp_write_to_pmem` and `sfp_write_to_kmem` flags.

The controller automatically generates status flags (e.g., `qmem_free`) to notify upper-level modules when memory banks are available for new data, enabling efficient cycle-saving data prefetching.

C. Sparsity-Aware Softmax (SFP) Engine

The SFP row block is responsible for normalizing one row of the MAC array outputs. The normalization process is divided into two pipelined phases:

1) **Accumulation (acc):** When `acc_start` is pulsed, the engine computes the row energy $S_i = \sum_j |sfp\_in[j]|$ using an adder tree. The sum is temporarily stored

in an internal FIFO, and `acc_done` is asserted upon completion.

2) **Division (div):** During the division phase, triggered by `div_start` (which is gated by a `div_busy` signal to prevent overlapping operations), the engine retrieves the local sum from the FIFO and adds it to the sum from the other core (`sum_in`). It then computes the normalized output:

$$sfp\_div\_out[c] = \text{trunc}\left(\frac{|sfp\_in[c]| \ll out\_shift}{S_{local} + sum\_in}\right) \quad (1)$$

Upon finishing the division, a `div_done` pulse is generated to indicate that the normalized results are valid.

To further optimize performance, a sparsity gating mechanism is implemented. If the computed row energy S_i is strictly less than a configurable `SFP_THRESHOLD`, the engine bypasses the division and outputs all zeros. This effectively filters out negligible attention scores, conserving dynamic power.

III. Design Methodology and Verification Infrastructure

To facilitate rapid development and reliable testing of the dual-core design, a robust Makefile-driven flow and shell-based verification infrastructure were established.

A. Makefile Management

A centralized Makefile provides unified commands for behavioral simulation, synthesis, and gate-level simulation (GLS) across various design targets (e.g., `fullchip`, `core`, `mac`, `sfp_row`). By standardizing the build system, team members could seamlessly run target-specific tests using commands such as `make sim TARGET=core` or `make syn TARGET=mac`, significantly improving development efficiency and avoiding configuration errors.

B. Shell-Based Verification and Automated Parsing

Synthesis quality of results (QoR) and timing closure were continuously tracked using a custom shell-based parser (`parse_reports.sh`). Integrated into the Makefile as the `make parse` target, this tool automatically scans Design Compiler output reports to extract and summarize critical metrics, including total cell area, dynamic power, leakage power, and Worst Negative Slack (WNS). This automated reporting mechanism proved essential during the RTL optimization phase (Step 5), allowing the team to quickly evaluate the impact of repipelining the SFP block on the overall critical path.

IV. Step-by-Step Implementation and Verification

To systematically conquer the design complexity, the project was executed in six sequential milestones.

A. Step 1: Single Core RTL, Synthesis & PNR

The baseline single-core design was implemented and verified first. This step established the core infrastructure, including the MAC array and the basic controller. Post-PnR layout, GLS waveform, and terminal output for the single-core design are shown below, with summary metrics in Table I.

To validate the flexibility of the single-core design, we separately tested the QK product and the V–Norm product using the same computation path. This demonstrates that the design can support both stages of the attention computation, providing a verified foundation for later steps where these operations are connected into a more complete pipeline.

In addition, minor modifications were made to the professor-provided testbench to align with our implementation. Specifically, we updated the DUT from fullchip to core, parameter settings, and data packing format, and added automatic golden result comparison (PASS/FAIL). These changes improve verification efficiency while preserving the intended computation functionality.

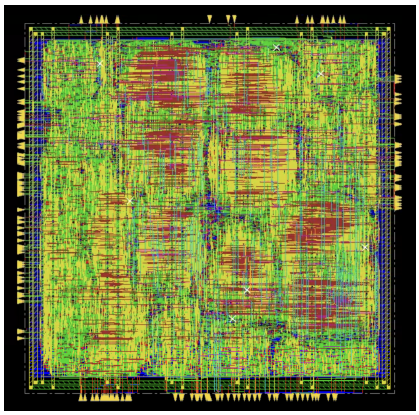

Fig. 1: Single-Core Layout (post-PnR).

```
xcelium> run
##### Q data txt reading #####
##### K data txt reading #####
##### Qmem writing #####
##### Kmem writing #####
##### K loading to MAC #####
##### Execute #####
QK phase verification
  [row]  RTL     col0 col1 col2 col3 col4 col5 col6 col7
[0] RTL:        93   -20    -7    94   -48   -45   -63   -86
    golden:     93   -20    -7    94   -48   -45   -63   -86
    [OK]

[1] RTL:        52   -21   -73    24    -5   -33   -41   -23
    golden:     52   -21   -73    24    -5   -33   -41   -23
    [OK]

[2] RTL:        38    27    50    63   -28    21    -3   -55
    golden:     38    27    50    63   -28    21    -3   -55
    [OK]

[3] RTL:        33   -74    17   125   -37   -17   -27   -51
    golden:     33   -74    17   125   -37   -17   -27   -51
    [OK]

[4] RTL:        80   -10   -10    13  -138   -37   -90   -56
    golden:     80   -10   -10    13  -138   -37   -90   -56
    [OK]

[5] RTL:        42     1    26    68     7     7    -8   -45
    golden:     42     1    26    68     7     7    -8   -45
    [OK]

[6] RTL:       -19    48    99    -7    -1    23    46   -20
    golden:    -19    48    99    -7    -1    23    46   -20
    [OK]

[7] RTL:       -63    41   128   -46    -1    78    74    37
    golden:    -63    41   128   -46    -1    78    74    37
    [OK]

------------------------------------------
PASS: all 8 x 8 results correct
------------------------------------------
Simulation complete via $finish(1) at time 111 NS + 0
 /netlist/stepl_tb.v:238     #10 $finish;
```

Fig. 2: Single-Core GLS Terminal Output.

B. Step 2: Output Normalization

The single-core MAC array was integrated with the SFP row block. Behavioral simulations verified that the accumulated

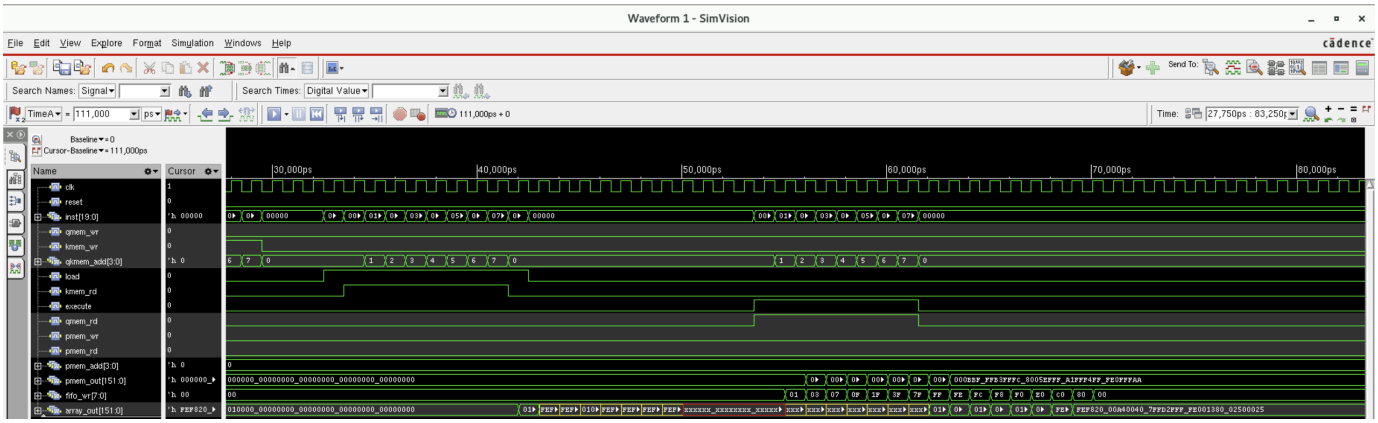

Fig. 3: Single-Core GLS Waveform.

TABLE I: Step 1 Synthesis & PnR Results

Design	Power (VCD)	WNS	DRC
Single Core	50.74 mW	-0.014 ns	0

sums were correctly utilized to normalize the partial sums. (Replace `temp.png` when the final waveform is finalized).

Fig. 4: Behavioral Simulation Waveform / TB for Output Normalization.

C. Step 3: Hierarchical Synthesis of Core

Memory macros (SRAMs for QMEM, KMEM, and PMEM) were incorporated into the core. A hierarchical synthesis and PnR flow was adopted to manage the physical placement of the SRAM macros alongside standard cells.

TABLE II: Step 3 Hierarchical Core Results

Design	Power (VCD)	WNS	DRC
Hier. Core	61.22 mW	—	—

D. Step 4: Hierarchical Synthesis of Dual-Core

The design was expanded to a full dual-core architecture. This step focused on non-lockstep execution across the two clock domains. Hierarchical PnR constraints were applied to seamlessly integrate both cores at the top level.

Fig. 5: SRAM_64b (Qmem & Kmem) Layout (placeholder).

Fig. 6: SRAM_160b (Psum_mem) Layout (placeholder).

E. Step 5: Alphas (Architecture Optimization)

Initial synthesis results highlighted a massive timing violation (Worst Negative Slack $\approx$ -23.9 ns) primarily due to deep combinational logic in the SFP row's absolute sum and division calculations. To achieve timing closure for a 1.0 ns clock period, aggressive RTL repipelining was implemented:

TABLE III: Step 4 Dual-Core Results

Design	Power (VCD)	WNS	DRC
Dual Core	19.19 mW	—	—

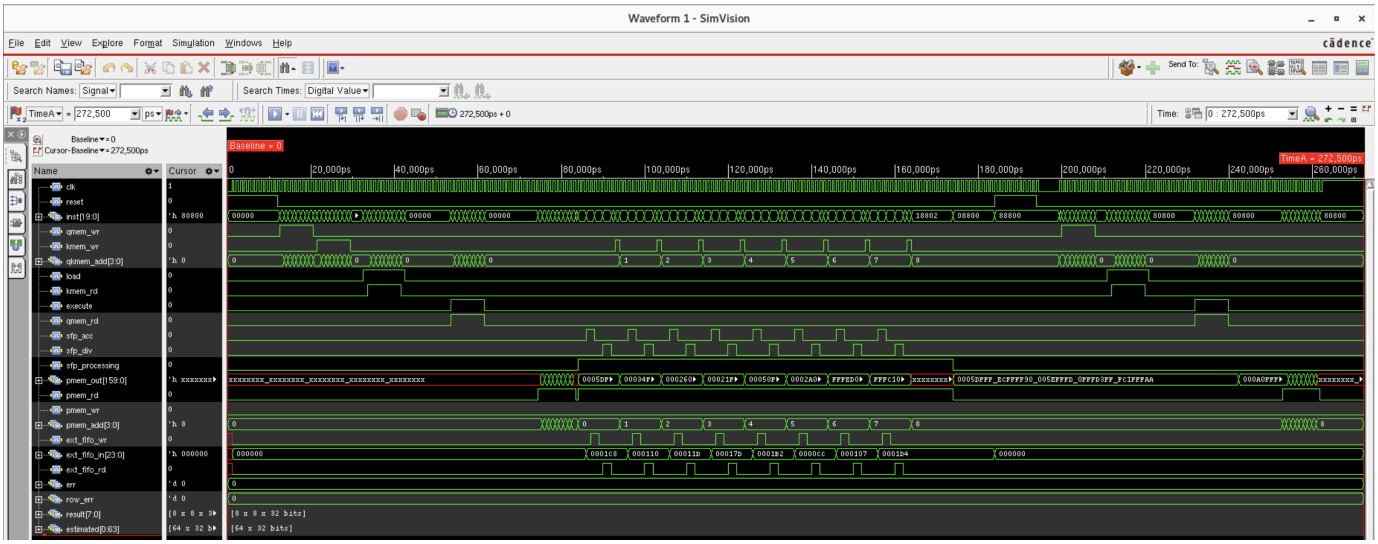

Fig. 8: Hierarchical Core GLS Waveform.

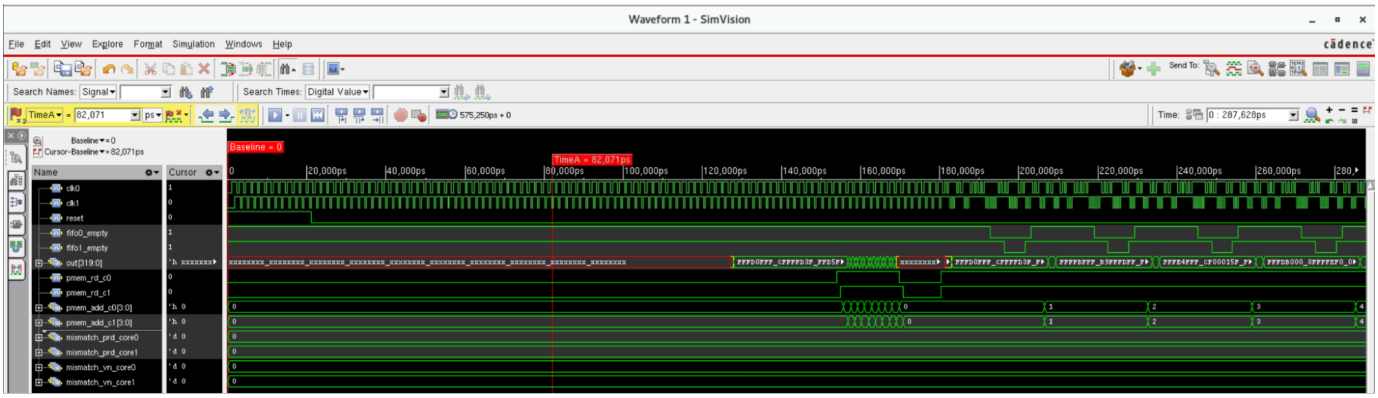

Fig. 11: Dual-Core GLS Waveform.

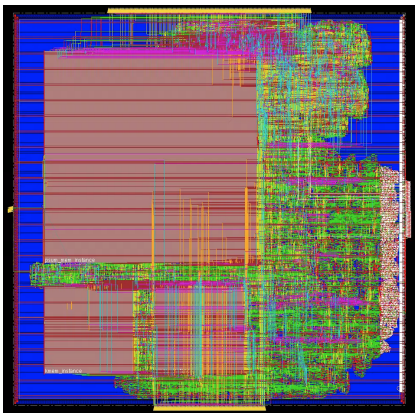

Fig. 7: Hierarchical Core Layout.

```
##### execute #####
VH phase verification start (checking pmem content)

##### sample pmem content & compare to golden #####
  [row]  RTL   :   col0   col1   col2   col3   col4   col5   col6   col7
         golden:   ----   ----   ----   ----   ----   ----   ----   ----

   [0]   RTL   :    160   -103    155    232   -311    283   -242    -79
         golden:    160   -103    155    232   -311    283   -242    -79
                [OK]

   [1]   RTL   :    238     85     98    127    334    179     54     75
         golden:    238     85     98    127    334    179     54     75
                [OK]

   [2]   RTL   :   -214   -333   -334     46   -430   -300   -281   -253
         golden:   -214   -333   -334     46   -430   -300   -281   -253
                [OK]

   [3]   RTL   :   -517   -433   -354   -314   -526   -524   -309   -381
         golden:   -517   -433   -354   -314   -526   -524   -309   -381
                [OK]

   [4]   RTL   :   -311     40   -233   -253   -196   -396    298     68
         golden:   -311     40   -233   -253   -196   -396    298     68
                [OK]

   [5]   RTL   :   -419   -414   -467   -366   -456   -465   -371   -432
         golden:   -419   -414   -467   -366   -456   -465   -371   -432
                [OK]

   [6]   RTL   :   -154    -21    -47    -73    140   -267    187      3
         golden:   -154    -21    -47    -73    140   -267    187      3
                [OK]

   [7]   RTL   :   -201     24    -35   -201   -351     42    -58    -38
         golden:   -201     24    -35   -201   -351     42    -58    -38
                [OK]

------------------------------------------------------------
  PASS  8 rows x 8 cols  all match estimated result
------------------------------------------------------------

Simulation complete via $finish(1) at time 272500 PS + 0
./netlist/core_tb.v:603     #10 $finish;
```

Fig. 9: Hierarchical Core GLS Terminal Output.

- **Divider Optimization:** The combinational divider was replaced with a multi-cycle long division FSM (`div_longdiv`).
- **Multi-cycle Path Constraint:** Multi-cycle path constraints were applied to the divider to correctly model its multi-cycle behavior during timing analysis , and the RTL design adjust accordingly by using a FSM to control multicycle path state. (`div_MCP`).
- **Adder Tree Optimization:** The row energy accumulator was split into a 2-stage pipeline (`sum8_2stage`).

Fig. 10: Fullchip Layout (placeholder; replace with fullchip snapshot).

```
================== OVERALL SUMMARY ====================
  Mode: NON-LOCKSTEP, split 40-bit inst bus
  clk0=1.000GHz (core0)  clk1=1.333GHz (core1)
  tick0 then tick1 per step - 1 posedge per core per operation
  QK phase -- CORE0: 0  CORE1: 0 mismatch(es)
  VN phase -- CORE0: 0  CORE1: 0 mismatch(es)
  >>> ALL TESTS PASSED <<<
=======================================================
Simulation complete via $finish(1) at time 575250 PS + 0
./netlist/fullchip_tb.v:714      #10 $finish;\r
xcelium>
```

Fig. 12: Dual-Core GLS Terminal Output.

These architectural optimizations successfully reduced the data arrival time from $\sim$25.0 ns down to 0.97 ns, eliminating all timing violations. Furthermore, the total cell area of the SFP row was reduced by 54% as the costly combinational divider was replaced.

TABLE IV: Timing Optimization Synthesis Results (Target Period = 1.0 ns)

Design	**Before Repipelining**		**After Repipelining**	
	Area (μm^2)	**WNS (ns)**	**Area (μm^2)**	**WNS (ns)**
Core	243,526	-23.68	175,820	+0.00
MAC Array	116,808	-1.61	69,490	+0.00
SFP Row	53,864	-23.92	24,698	+0.00

F. Step 6: Sparsity-Aware Attention

The final milestone validated the sparsity-aware feature. By injecting highly sparse matrices, we confirmed that the SFP engine correctly identifies row energies below `SFP_THRESHOLD` and outputs zeros, successfully bypassing unnecessary long division cycles.

V. Conclusion

This project successfully demonstrates the design, implementation, and physical realization of a dual-core attention accelerator. By partitioning the system into asynchronous domains, we enabled flexible inter-core data exchange. Through the introduction of a Sparsity-Aware SFP engine, computations on insignificant data are bypassed, promoting energy efficiency. Most importantly, critical timing bottlenecks were identified and resolved via rigorous RTL repipelining, ensuring the design easily meets the 1.0 ns clock constraint in a TSMC 65nm process. The progressive, step-by-step verification methodology guaranteed robustness from the behavioral level down to post-PnR gate-level simulations, confirming the design's readiness for physical fabrication.

Fig. 13: Sparsity-Aware Attention Dual-Core Layout (placeholder).

Fig. 15: Sparsity-Aware Attention Dual-Core GLS Terminal Output (placeholder).

References

[1] E. Course, "Ece260b final project (placeholder)," Course project report, 2026, replace with real references; use cite in main.tex to list them.

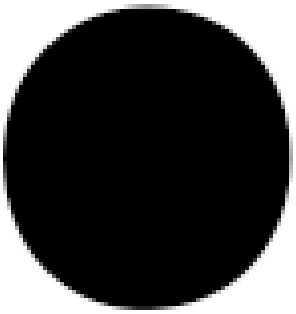

Fig. 14: Sparsity-Aware Attention Dual-Core GLS Waveform (placeholder).